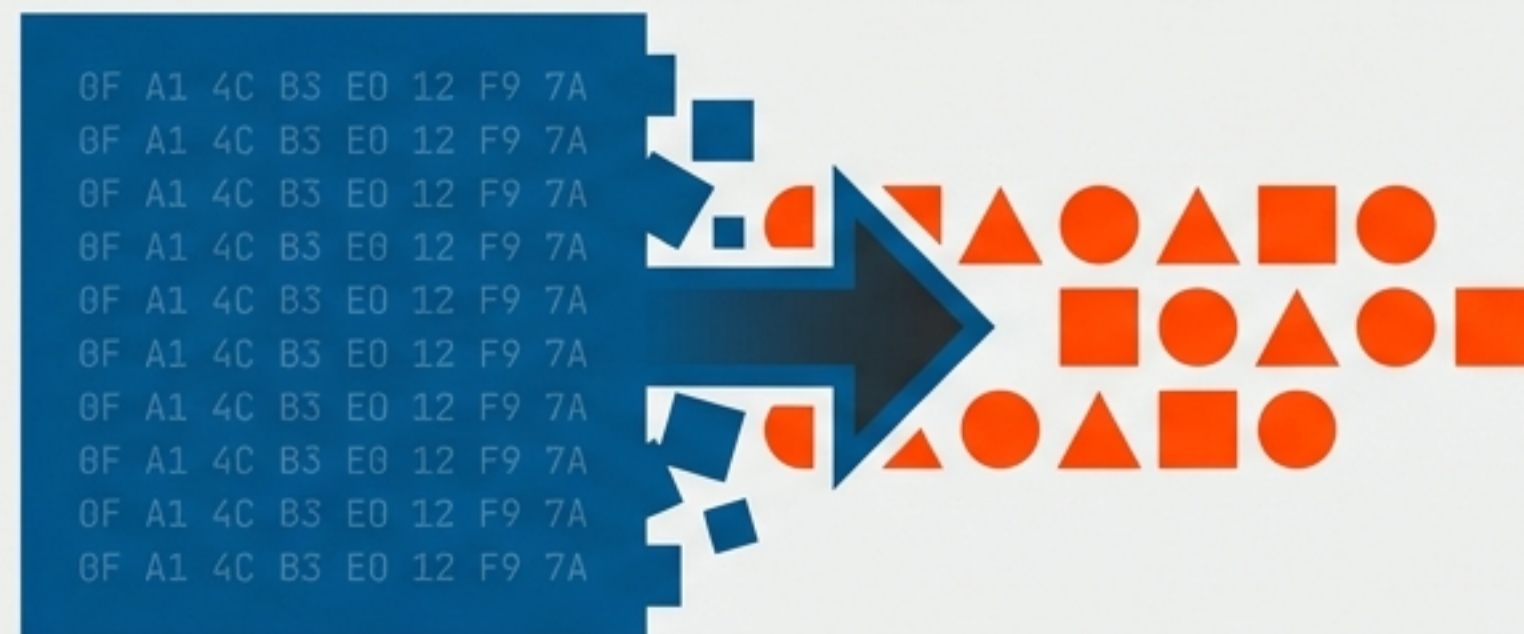


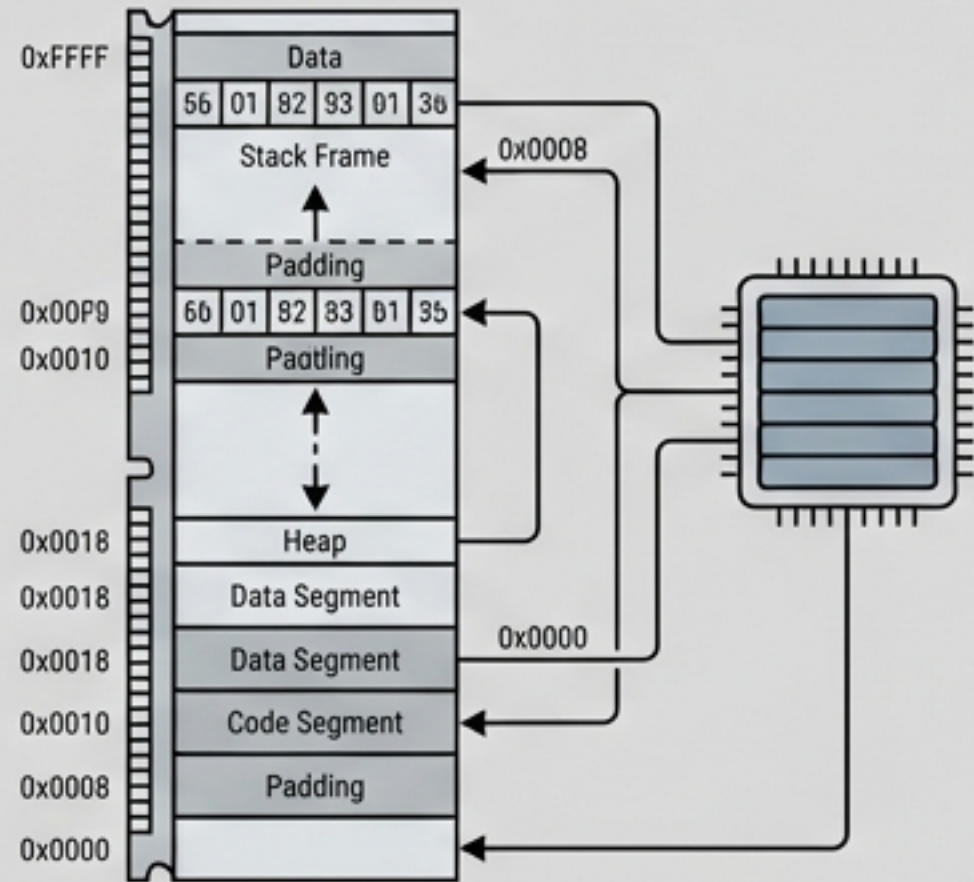
Серіалізація та Десеріалізація в Системному Програмуванні

Від побайтового леяуту до
зовнішніх контрактів



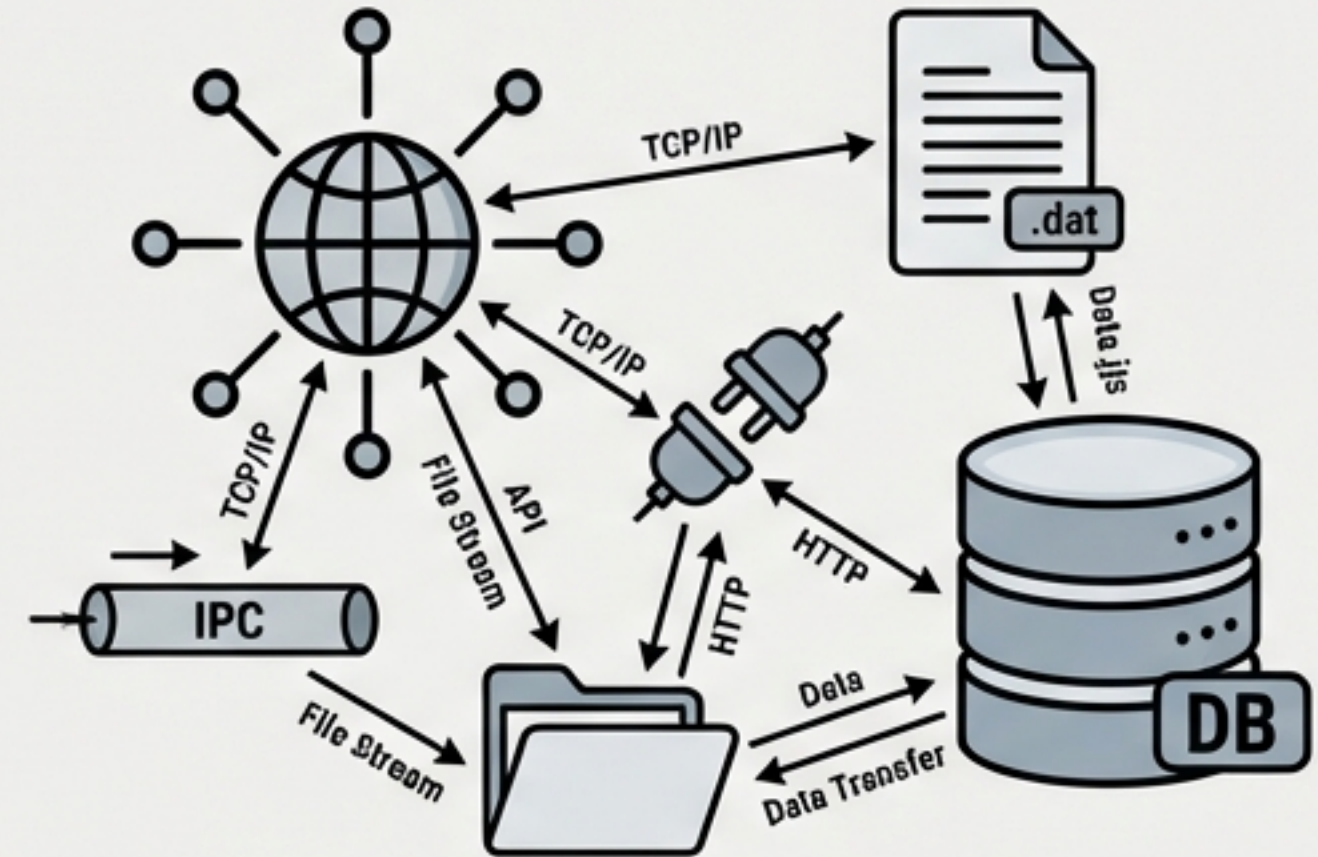
Задачі, які нам потрібно вирішувати

Пам'ять процесу (Internal Layout)



- Байти, вирівнювання, адреси (Pointers)
- Залежність від Endianness та ABI
- Оптимізація під CPU

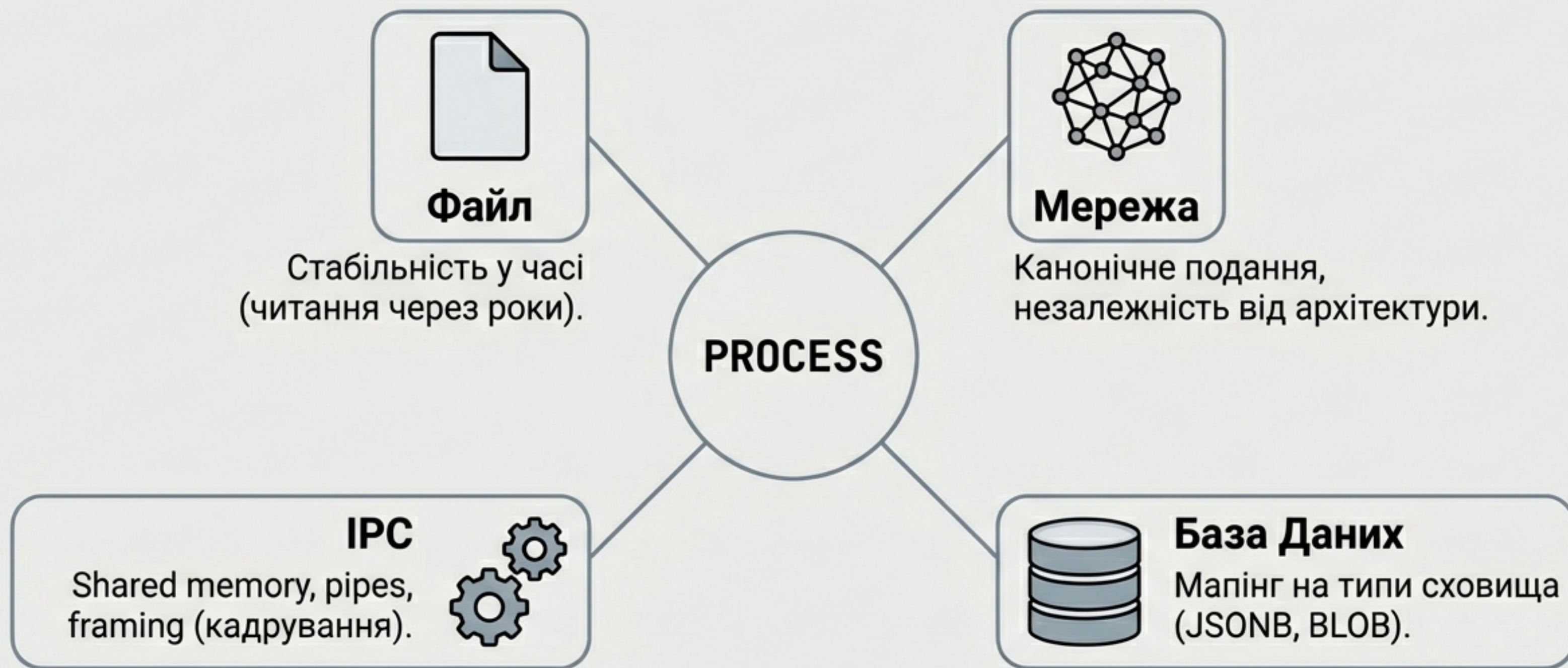
Зовнішнє середовище (External Contract)



- Файли, Сокети, IPC
- Вимога переносимості та стабільності
- Оптимізація під транспорт/зберігання

**Просто записати struct у файл — це некоректна постановка задачі.
Нам потрібен формат.**

Приклади задач та канали даних

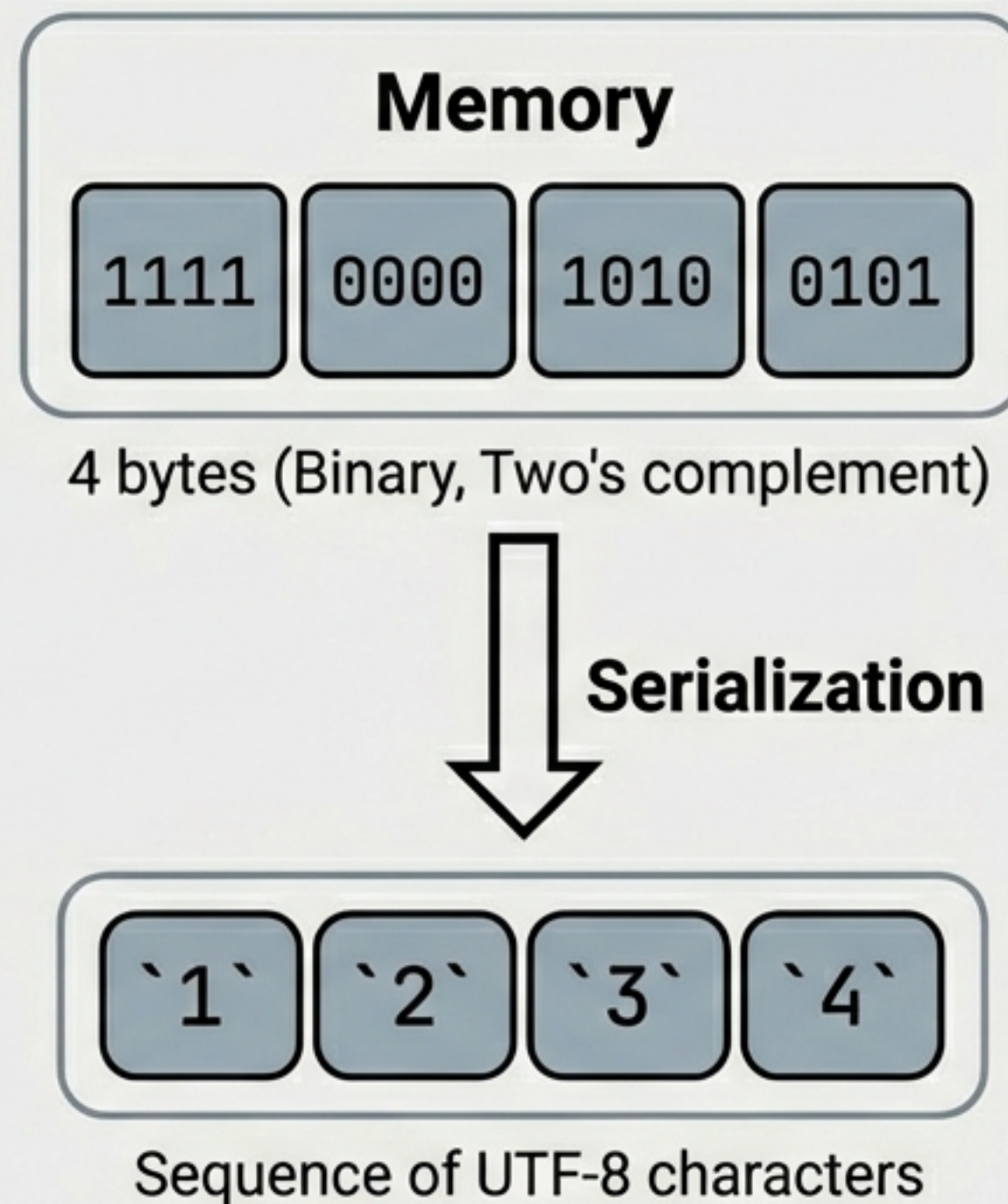


Output (Serialization): Safe context → External format

Input (Deserialization): Untrusted source → Validation → Safe context

Друк на консоль: Перший крок

```
fn convert(x: i32) -> String {  
    // Implementation hidden  
}  
  
// Usage:  
println!("{}", x);
```



Display: Human readability (unstable)

Serialization: Data exchange (strict contract)

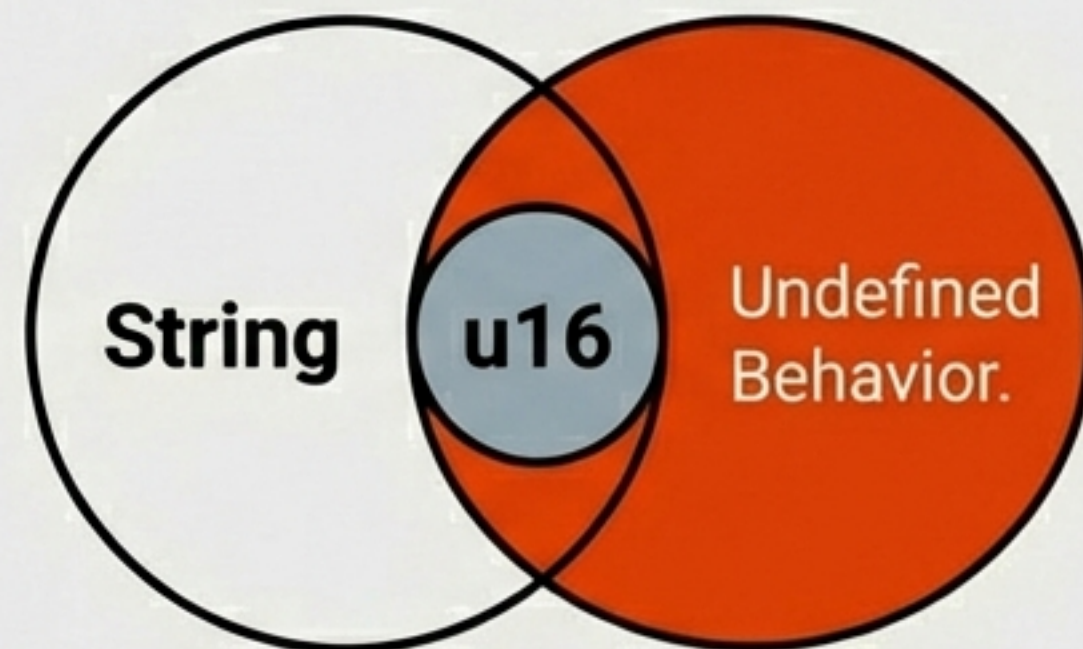
Читання з консолі: Пастка наївності

```
fn convert(raw: String) -> u16 { ✗  
  
  // What happens if raw is "hello"?  
  // What happens if raw is "70000"?  
  // What happens if raw is ""?
```

Сигнатура – це брехня.

Часткова функція (Partial Function)

- ! panic
- ! некоректна логіка
- ! порушення інваріантів

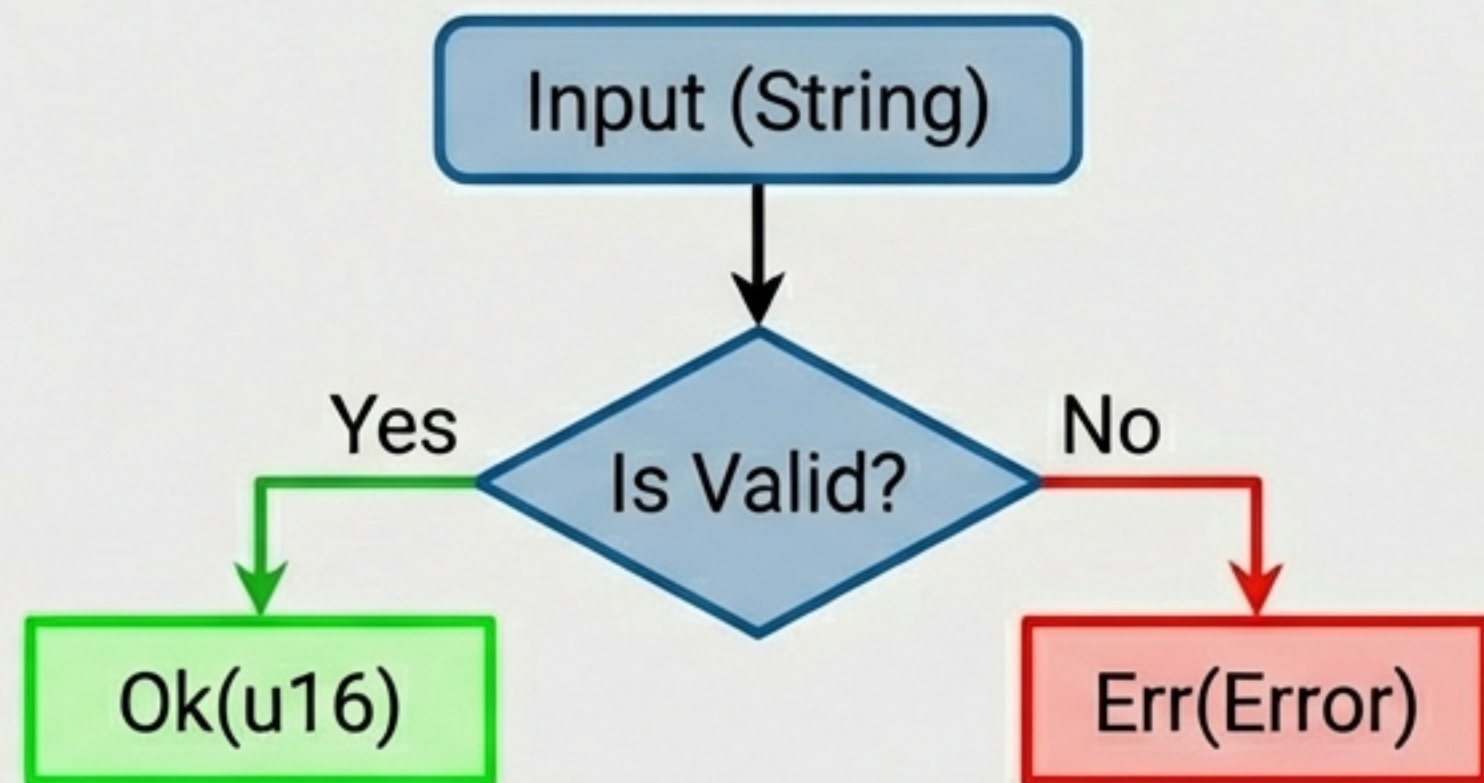


Output (Serialization): Safe context → External format

Input (Deserialization): Untrusted source → Validation → Safe context

Читання з консолі: Чесна функція

```
fn convert(raw: String) ->  
    Result<u16, Error> {  
    // Returns Ok(u16) or Err(Error)  
}
```



- **Type Safety:** Помилка стає частиною контракту.
- **Control Flow:** Заміна `panic` на обробку гілок.
- **Fail-Fast:** I/O відокремлено від бізнес-логіки.

Output (Serialization): Safe context → External format

Input (Deserialization): Untrusted source → Validation → Safe context

Читання з консолі: Rust Approach

```
// The Standard Trait
impl FromStr for u16 {
    type Err = ParseIntError;
    fn from_str(s: &str) -> Result<Self, Self::Err> {
        // Implementation...
    }
}

// Idiomatic usage
let x: u16 = s.parse()?;
```

FromStr

Стандартизація інтерфейсу десеріалізації.

Operator ?

Лінійна обробка помилок (early return).

Composability

Ланцюжки обробки ('unwrap', 'expect', 'or_else')

Якщо структура складна

```
struct Person {  
    id: u32,  
    name: String,  
}  
  
// Naive format: "id:name"  
// Example: "1:John"
```

Що робити, якщо в імені є двокрапка? (Escaping)

1:John:Doe



- **Ordering:** Порядок полів має значення.
- **Validation:** $id > 0$, name не порожнє.

Текстовий формат — це міні-протокол.

Бінарний файл (Binary Format)

```
fn to_bytes(&self) -> Vec<u8> {  
    // ...  
}
```

// Returns raw byte buffer

Raw binary representation.

	04	A1	FF	00	00	C9	2E	3B	7B	77	DB	DB	DB	E4	
04	04	A1	FF	00	00	C9	A1	FF	00	04	A1	FF	00	00	
04	04	A1	FF	00	00	FF	00	2E	2E	3B	7B	77	DB	E4	
	04	00	00	00	00	C9	00	00	00	00	00	00	00	8B	3B
3B	04	A1	FF	00	00	C9	2E	2E	2B	3B	77	DB	DB	E4	
	2E	3B	7B	77	E8	E4	2E	2E	3B	3B	77	DB	DB	E4	04
C9	2E	3B	77	DB	E4	E8	E8	2E	2E	3B	7B	77	DB	E3	
	E4	2E	2E	3B	77	DB	E4	3B	7B	D9	27	07	DB	E4	3B
04	A1	3B	77	DB	E4	00	00	0B	04	04	A1	FF	00	00	
04	FF	00	00	00	FF	00	00	04	04	A1	A2	FF	01	8B	04
	0B	E4	04	02	E8	00	06	AA	FF	FF	00	00	E9	00	

✓ Pros

- Компактність, швидкість, відсутність парсингу.

✗ Cons

- Крихкість (один байт ламає все).
- Залежність від Endianness.
- Проблема динамічних даних (String на heap).

Use Case: High Load, Network Protocols.

JSON: Універсальний стандарт

```
{
```

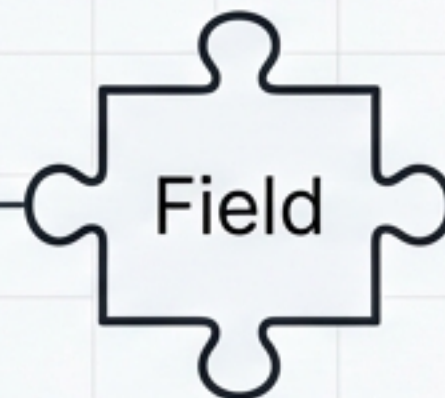
```
  "id": 1,
```

```
  "name": "John"
```

```
}
```

Rust `serde` crate

"name"

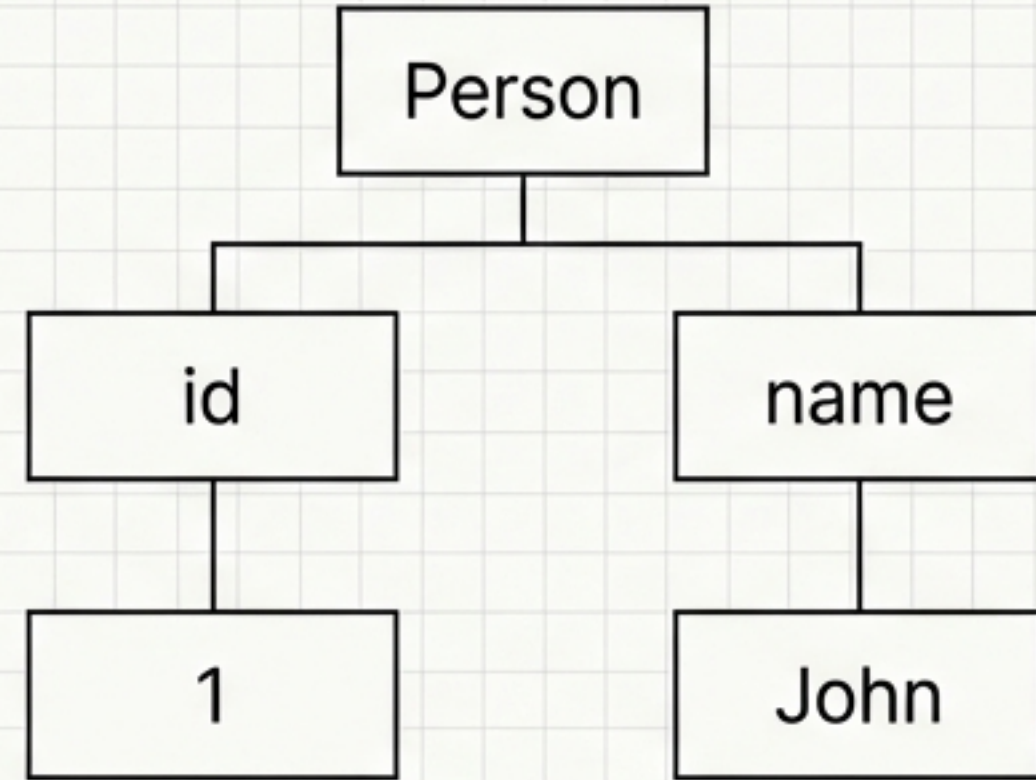


- **Self-describing:** Поля мапляться за іменем.
- **Interoperability:** Зрозуміло для JS, Python, Go.
- **Trade-off:** Текстовий оверхед та вартість парсингу.
- **Contract:** Структура JSON стає частиною публічного API.

XML: Документна модель

```
<Person>
  <id>1</id>
  <name>John</name>
</Person>
```

XML Document Structure.



- **Nature:** Data as a Document (DOM).
- **Validation:** XSD дозволяє формальну перевірку типів.
- **Drawbacks:** Verbosity (теги), складний парсинг.
- **Context:** Enterprise, Legacy, суворі контракти.